



ECOLE NATIONALE SUPÉRIEURE DE TECHNIQUES AVANCÉES

ENSTA 2^E ANNÉE

COURS APM 4RO03

Filling

Serena DAGHER Yara EL CHAM

Jeux et R.O

8 Mai 2026

Lien Code source

Table des matières

1	Présentation du jeu <i>Filling</i>	2
1.1	Règles du jeu	2
1.2	Représentation des instances	2
1.3	Objectif du projet	3
2	Formulation mathématique du problème	3
2.1	Variables de décision	3
2.2	Fonction objectif	3
2.3	Contraintes principales	3
3	Implémentation en Julia	5
3.1	Structure du projet	5
3.2	Lecture des instances	5
3.3	Affichage de la grille et de la solution	5
3.4	Résolution exacte avec CPLEX	6
3.5	Callback pour la règle difficile	6
3.6	Fonction de voisinage	7
3.7	Vérification d'une solution	7
3.8	Heuristiques testées	7
3.9	Génération d'instances	8
3.10	Résolution du dataset	8
4	Résultats	9
4.1	Résolution des instances	9
4.2	Graphiques	10
4.3	Test sur une grille 20×20	11
4.4	Analyse	12
4.5	Observations	13
5	Conclusion	13

1 Présentation du jeu *Filling*

1.1 Règles du jeu

Filling (difficulté 8) est un puzzle logique sur une grille $n \times n$. Le but est de remplir toutes les cases vides avec des chiffres.

Les règles sont les suivantes :

- Toute la grille doit être divisée en blocs.
- Chaque bloc contient des cases adjacentes horizontalement ou verticalement.
- Toutes les cases d'un même bloc contiennent le même chiffre.
- Si un bloc contient le chiffre x , alors ce bloc doit contenir exactement x cases.
- Les chiffres déjà présents dans la grille sont fixés et doivent être conservés.
- Les cases vides peuvent créer de nouveaux blocs, même si ces blocs ne contiennent aucun chiffre donné au départ.
- Deux blocs de même taille ne peuvent pas se toucher horizontalement ou verticalement.

Dans notre modélisation, on ne prend pas en compte les murs affichés par le jeu. Ils séparent seulement des blocs de valeurs différentes et ne changent pas les règles principales de résolution. On se concentre donc sur la connexité des blocs et sur leur taille.

1.2 Représentation des instances

Chaque instance du jeu *Filling* est stockée dans un fichier texte avec le format suivant :

- Chaque ligne correspond à une ligne de la grille.
- Les cases sont séparées par des virgules.
- 0 : case vide.
- 1..n : chiffre pré-rempli.

Exemple : grille 6×6

```
1, 0, 0, 0, 6, 0
0, 0, 4, 0, 0, 0
0, 2, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
0, 0, 4, 5, 0, 0
5, 0, 4, 4, 0, 0
```

La grille contient donc uniquement les chiffres donnés et les cases vides. Les murs ne sont pas encodés dans les instances utilisées par notre solveur.

1.3 Objectif du projet

- Modéliser le jeu avec un programme linéaire en nombres entiers (PLNE).
- Développer une heuristique pour les grandes instances.
- Générer des instances à partir de solutions valides connues et résoudre un dataset complet.
- Comparer les performances de CPLEX et de l'heuristique selon la taille de la grille.
- Analyser la faisabilité et visualiser les résultats par taille de grille.

2 Formulation mathématique du problème

2.1 Variables de décision

Pour chaque case $v \in V$ et chaque valeur possible k ($1 \leq k \leq K$) :

$$x_{v,k} = \begin{cases} 1 & \text{si la case } v \text{ prend la valeur } k, \\ 0 & \text{sinon.} \end{cases}$$

où K correspond à la valeur maximale autorisée dans le modèle, notée `maxVal` dans le code.

2.2 Fonction objectif

Comme le but principal est de trouver une solution réalisable, nous fixons :

$$\min \quad 0$$

2.3 Contraintes principales

1. **Chaque case prend exactement une valeur.**

Pour chaque case v , on impose :

$$\sum_{k=1}^K x_{v,k} = 1$$

Ainsi, chaque case reçoit une seule valeur.

Dans le code, cela correspond à :

```
1 for i in 1:n, j in 1:m
2     @constraint(m_model, sum(x[i,j,k] for k in 1:maxVal) == 1)
3 end
```

Cette contrainte permet aussi aux cases vides de créer de nouveaux blocs : si $\text{grid}[i,j] = 0$, aucune valeur n'est imposée au départ, donc le solveur peut choisir librement une valeur entre 1 et K .

2. Les cases préremplies gardent leur valeur.

Si une case v contient initialement une valeur $g_v > 0$, alors cette valeur est fixée dans le modèle :

$$x_{v,g_v} = 1$$

Cette contrainte garantit que les chiffres donnés dans l'instance ne peuvent pas être modifiés par le solveur. Comme chaque case doit prendre exactement une seule valeur, fixer x_{v,g_v} à 1 force automatiquement toutes les autres variables $x_{v,k}$ de cette case à 0.

Dans le code, cela correspond à :

```

1 for i in 1:n, j in 1:m
2     if grid[i,j] > 0
3         @constraint(m_model, x[i,j,grid[i,j]] == 1)
4     end
5 end

```

3. Chaque région de valeur k doit être connexe et de taille k .

Cette contrainte est difficile à écrire entièrement dès le début du modèle, car il faudrait prévoir toutes les composantes possibles. Nous utilisons donc un callback CPLEX avec des contraintes paresseuses.

Lorsqu'une solution entière est trouvée par CPLEX, le callback analyse les cases ayant la même valeur k . Il construit les composantes connexes à l'aide d'un parcours en largeur (BFS), en utilisant les voisins horizontaux et verticaux de chaque case.

Pour chaque composante de valeur k :

- si sa taille est exactement k , elle est correcte ;
- si sa taille est supérieure à k , on ajoute une contrainte qui interdit de reprendre exactement cette même composante ;
- si sa taille est inférieure à k , on ajoute une contrainte qui force la composante à changer ou à s'étendre.

Cette contrainte interdit aussi automatiquement deux blocs de même taille qui se touchent. En effet, deux blocs de valeur k qui se touchent deviennent une seule composante connexe trop grande.

4. Les variables sont binaires.

$$x_{v,k} \in \{0, 1\}$$

3 Implémentation en Julia

3.1 Structure du projet

Comme demandé dans le sujet, le projet est organisé en plusieurs fichiers :

- `main.jl` : point d'entrée du projet. Il permet de lancer les tests et la résolution du dataset.
- `src/io.jl` : lecture des instances, affichage des grilles et des solutions.
- `src/resolution.jl` : résolution avec CPLEX, callback et heuristique.
- `src/generation.jl` : génération des instances et création du dataset.
- `src/solutions.jl` : contient des solutions complètes valides pour plusieurs tailles de grilles.

On utilise **JuMP** pour écrire le modèle PLNE et **CPLEX** comme solveur exact.

3.2 Lecture des instances

La fonction `readInputFile(path)` lit une instance depuis un fichier texte.

Le fichier contient uniquement la grille. La grille est stockée dans une matrice `grid`. Les cases vides sont représentées par 0 et les cases pré-remplies par leur valeur.

La fonction retourne donc :

```
n, m, grid
```

où :

- `n` et `m` sont les dimensions de la grille ;
- `grid[i,j]` contient la valeur de la case (i,j) ;
- une valeur 0 signifie que la case est vide.

3.3 Affichage de la grille et de la solution

Nous avons complété les fonctions :

- `displayGrid(n, m, grid)` : affiche une instance non résolue.
- `displaySolution(n, m, sol)` : affiche une solution trouvée.

Ces fonctions permettent de visualiser facilement la grille initiale et la solution retournée par CPLEX.

3.4 Résolution exacte avec CPLEX

La fonction principale de résolution est :

```
cplexSolve(n, m, grid)
```

Elle construit le modèle PLNE avec les variables binaires :

$$x_{i,j,k} = \begin{cases} 1 & \text{si la case } (i,j) \text{ prend la valeur } k, \\ 0 & \text{sinon.} \end{cases}$$

Elle construit ensuite les contraintes décrites dans la section précédente.

3.5 Callback pour la règle difficile

La contrainte de connexité est la plus difficile à écrire directement dans le modèle. Pour cette raison, elle est traitée avec un callback CPLEX. Le callback n'est appelé que lorsque CPLEX propose une solution entière. Il récupère alors les valeurs des variables, vérifie les composantes connexes de chaque valeur, puis ajoute une contrainte paresseuse uniquement si une composante n'a pas la bonne taille. Ainsi, on évite d'ajouter toutes les contraintes de connexité dès le départ, ce qui permet de garder un modèle PLNE plus petit.

```
1 function callback_filling(cb_data::CPLEX.CallbackContext, context_id::
    Clong)
2     if isIntegerPoint(cb_data, context_id)
3         CPLEX.load_callback_variable_primal(cb_data, context_id)
4         x_val = callback_value.(cb_data, x)
5         # Verification des composantes connexes
6         # et ajout de contraintes si necessaire
7     end
8 end
9 MOI.set(m_model, CPLEX.CallbackFunction(), callback_filling)
10 MOI.set(m_model, MOI.NumberOfThreads(), 1)
```

Exemple de coupe ajoutée

Supposons que CPLEX propose la solution suivante, où les cases (1,1) et (1,2) ont toutes les deux la valeur 3, mais sans aucun voisin de valeur 3 :

3	3	1
1	1	1
1	1	1

Le BFS trouve la composante $C = \{(1,1), (1,2)\}$ de taille $2 \neq k = 3$. Le callback ajoute alors la contrainte :

$$x_{1,1,3} + x_{1,2,3} \leq 1 + x_{2,1,3} + x_{2,2,3} + x_{1,3,3}$$

Cette règle dit : les cases (1,1) et (1,2) ne peuvent être simultanément de valeur 3 que si au moins un de leurs voisins l'est aussi .

Cette unique contrainte élimine d'un coup **toutes** les grilles où $(1, 1) = 3$ et $(1, 2) = 3$ mais aucun voisin n'est 3, peu importe les valeurs des autres cases. C'est pourquoi le callback est efficace : chaque coupe supprime une famille entière de mauvaises solutions sans avoir à les tester une par une.

3.6 Fonction de voisinage

Pour vérifier la connexité des blocs, on doit connaître les voisins de chaque case. Nous avons donc utilisé une fonction de voisinage qui retourne les cases adjacentes horizontalement et verticalement.

```
getNeighbors(n, m, i, j)
```

Cette fonction regarde les quatre directions : haut, bas, gauche et droite. Elle ne renvoie que les cases qui restent à l'intérieur de la grille. C'est cette fonction qui est utilisée dans le callback et dans la vérification des solutions pour construire les composantes connexes.

3.7 Vérification d'une solution

Nous avons aussi utilisé une fonction de vérification, appelée par exemple :

```
checkSolution(n, m, sol)
```

Elle vérifie que la solution respecte bien la règle du jeu :

- chaque case contient un chiffre ;
- pour chaque chiffre k , chaque zone connexe de k contient exactement k cases ;
- deux zones de même taille ne se touchent pas, car sinon elles formeraient une seule composante trop grande.

Cette fonction est utile pour vérifier les solutions obtenues par CPLEX, mais aussi pour tester les heuristiques.

3.8 Heuristiques testées

Comme le jeu *Filling* est de difficulté élevée, nous avons aussi testé plusieurs heuristiques pour comparer avec la méthode exacte CPLEX.

Au début, nous avons essayé une heuristique très simple qui remplissait les cases une par une avec des choix locaux. Cette méthode échouait souvent, car un mauvais choix au début pouvait bloquer la grille plus loin.

Nous avons aussi essayé une approche par construction directe de zones : l'idée était de former des régions connexes de taille k avec une BFS. Cette méthode était plus proche de la règle du jeu, mais elle restait trop gloutonne : une région construite trop tôt pouvait empêcher de compléter correctement les autres régions.

Finalement, nous avons adopté une heuristique avec **backtracking** et vérifications locales. À chaque étape, on choisit une case vide parmi les plus contraintes, c'est-à-dire une case qui a beaucoup de voisins déjà remplis. Ensuite, on teste d'abord les valeurs présentes autour de cette case, afin d'essayer de prolonger des régions existantes.

Après chaque placement, on vérifie que la région reste possible :

- sa taille ne doit pas dépasser sa valeur ;
- si elle est encore trop petite, il doit rester assez de cases libres accessibles pour pouvoir l'agrandir ;
- les régions voisines sont aussi vérifiées.

Si un choix rend la grille impossible, l'heuristique revient en arrière et essaie une autre valeur. Une limite de temps est fixée pour éviter une recherche trop longue.

À la fin, la solution obtenue est vérifiée avec **checkSolution**. Cette heuristique n'est pas garantie de trouver une solution, contrairement à CPLEX, mais elle est plus robuste que les premières heuristiques gloutonnes testées.

3.9 Génération d'instances

Au début, nous avons essayé de générer automatiquement des solutions complètes faisables avec du code, puis de cacher certaines valeurs pour obtenir des instances. Cependant, cette méthode ne donnait pas toujours des solutions valides : certaines régions de même valeur pouvaient se toucher et former une composante trop grande.

Nous avons donc choisi une méthode plus fiable. Nous avons ajouté un fichier **solutions.jl** qui contient des solutions complètes valides pour plusieurs tailles de grilles. La génération d'une instance se fait maintenant à partir de ces solutions connues.

La fonction **generateInstance(n,m)** récupère d'abord la solution complète correspondant à la taille demandée :

```
1 sol = KNOWN_SOLUTIONS[(n, m)]
```

Ensuite, elle crée une grille vide remplie de 0. Pour chaque région connexe de la solution, on révèle une seule case comme indice. Les autres cases restent vides.

Ainsi, chaque instance générée est construite à partir d'une solution complète déjà valide. Cela garantit que l'instance possède au moins une solution.

Dans nos tests, nous générons 50 instances pour chaque taille disponible, de 3x3 à 12x12.

3.10 Résolution du dataset

Enfin, la fonction :

```
solveDataSet()
```

résout toutes les instances du dossier `data/`. Pour chaque instance, elle lance CPLEX et l’heuristique, puis elle écrit les résultats dans le dossier `res/`.

Chaque fichier de résultat contient :

```
solveTime = ...
SolutionFound = ...
```

Pour l’heuristique, on ajoute aussi si la solution obtenue est valide ou non. Cela permet ensuite de construire le tableau de résultats.

4 Résultats

4.1 Résolution des instances

Les instances du dataset sont générées à partir de solutions complètes connues. Elles possèdent donc au moins une solution valide.

Pour chaque taille, nous avons généré plusieurs instances en révélant une case par région de la solution complète. Les instances d’une même taille peuvent donc être différentes, car la case révélée dans chaque région est choisie aléatoirement.

Le tableau suivant présente les résultats obtenus avec CPLEX et avec l’heuristique. Le nombre d’instances testées n’est pas toujours le même pour les deux méthodes, car certaines expériences ont été lancées séparément.

Pour CPLEX, nous avons fixé une limite de temps de 1 minute par instance. Pour l’heuristique, la limite de temps est de 10 secondes par instance. Nous nous sommes arrêtés à la taille 11×11 pour l’heuristique, car elle ne trouvait presque plus de solutions rapidement sur les grandes grilles.

Taille	Tests CPLEX	Résolues CPLEX	Temps moyen CPLEX (s)	Résolues heuristique	Temps moyen heuristique (s)
3×3	50	50	0.009	50 / 50	0.007
4×4	50	50	0.009	7 / 50	0.0003
5×5	50	50	0.024	39 / 50	0.022
6×6	50	50	0.095	40 / 50	1.334
7×7	50	50	0.256	12 / 50	1.706
8×8	50	50	0.253	4 / 50	2.965
9×9	50	49	2.796	5 / 50	2.129
10×10	50	50	4.866	2 / 50	1.471
11×11	14	14	10.390	0 / 46	—
12×12	50	32	14.212	—	—

Table 1: Comparaison entre CPLEX et l’heuristique sur le dataset généré

Les temps moyens sont calculés uniquement sur les instances résolues par la méthode concernée. Les instances arrêtées par limite de temps ne sont donc pas incluses dans ces moyennes. Le symbole “—” signifie que les résultats n’ont pas été calculés ou qu’aucune instance n’a été résolue pour cette taille.

4.2 Graphiques

La figure 1 montre le nombre d'instances résolues par CPLEX selon la taille de la grille. On observe que toutes les instances sont résolues jusqu'à la taille 8×8 . CPLEX reste aussi efficace sur les instances 10×10 testées, mais le temps de calcul augmente. Pour la taille 12×12 , plusieurs instances atteignent la limite de temps.

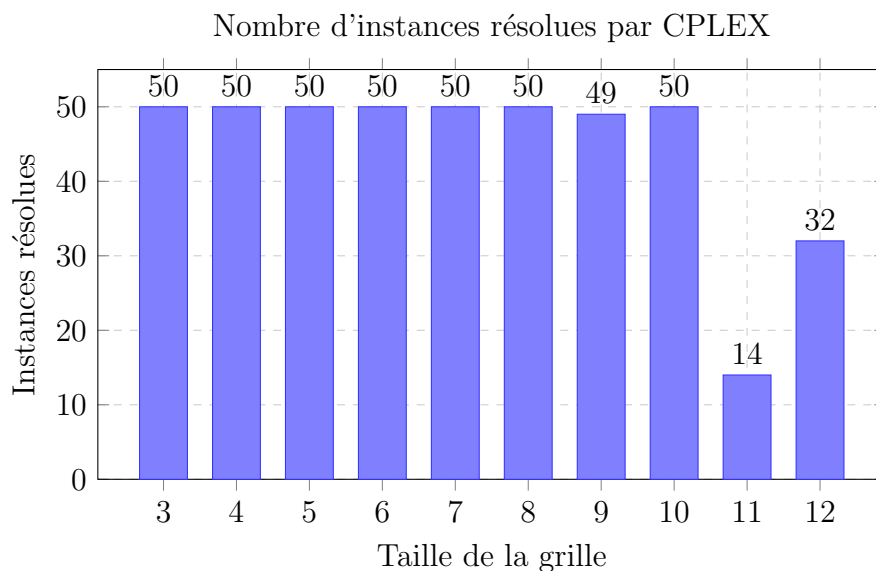


Figure 1: Nombre d'instances résolues par CPLEX selon la taille

La figure 2 montre le temps moyen de résolution pour les instances résolues par CPLEX. Le temps augmente fortement lorsque la taille de la grille augmente, surtout à partir de 9×9 .

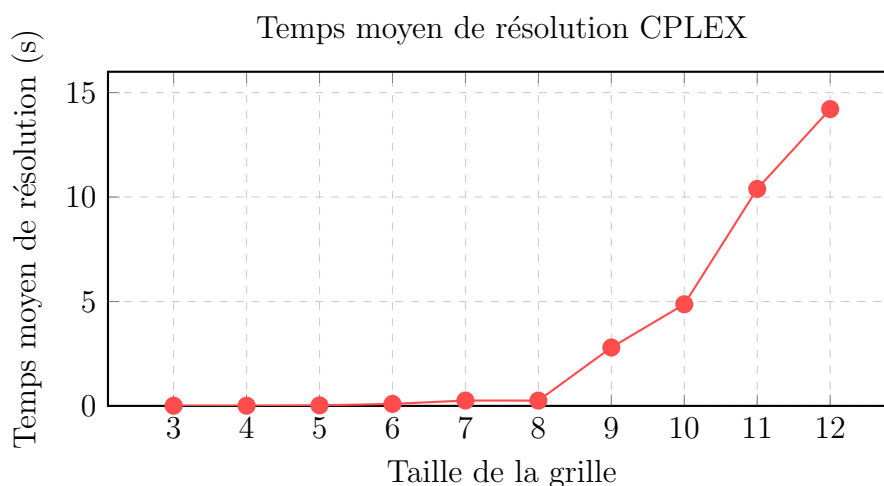


Figure 2: Temps moyen de résolution CPLEX en fonction de la taille

La figure 3 compare le nombre d'instances résolues par CPLEX et par l'heuristique. On voit que l'heuristique fonctionne bien sur certaines petites tailles, mais devient moins fiable lorsque la grille grandit.

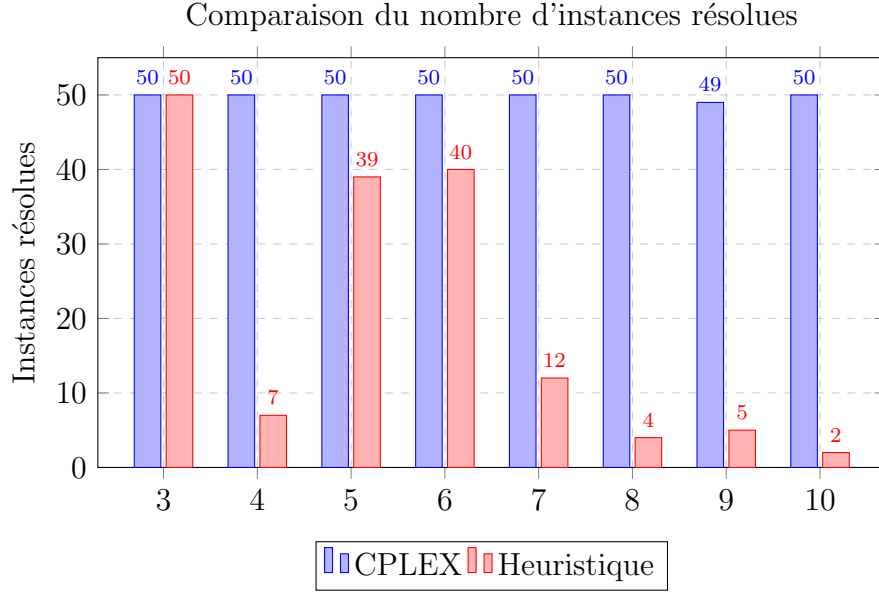


Figure 3: Comparaison entre CPLEX et l'heuristique

4.3 Test sur une grille 20×20

En plus du dataset principal, nous avons aussi testé le modèle sur une vraie grille de taille 20×20 .

Sur cette instance, CPLEX a réussi à trouver une solution valide, tandis que l'heuristique n'a pas réussi à résoudre la grille. Ce test confirme que le modèle PLNE avec callback reste capable de résoudre une grande grille réelle, même lorsque l'heuristique échoue.

La figure 4 montre, à gauche, la solution obtenue par CPLEX, et à droite, la solution affichée par le jeu.

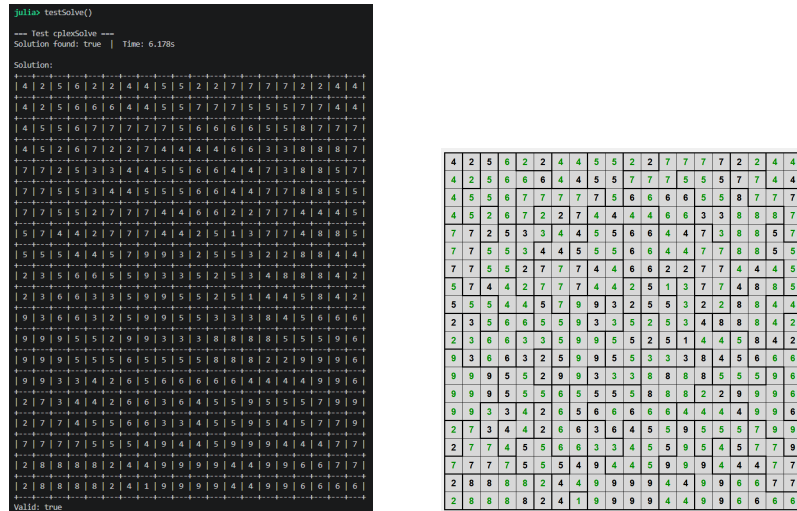


Figure 4: Grille 20×20 : à gauche, solution obtenue par CPLEX ; à droite, solution du jeu.

Même si la solution trouvée par CPLEX peut être différente de celle affichée par le

jeu, cela ne pose pas de problème. Une grille de *Filling* peut avoir plusieurs solutions valides. L'important est que la solution respecte bien toutes les règles du jeu.

4.4 Analyse

Une solution est-elle toujours trouvée ? Pour les petites tailles, CPLEX trouve toujours une solution. Toutes les instances de tailles 3×3 à 8×8 sont résolues. Pour la taille 9×9 , CPLEX résout 49 instances sur 50, et pour la taille 10×10 , les 50 instances testées sont résolues.

Pour 12×12 , CPLEX résout 32 instances sur 50. Les instances non résolues ne sont pas forcément infaisables, car elles sont générées à partir de solutions connues. Elles correspondent plutôt à des cas où CPLEX atteint la limite de temps.

Nous avons aussi testé une vraie grille 20×20 . Sur cette instance, CPLEX a trouvé une solution valide, ce qui montre qu'il peut encore résoudre de grandes grilles lorsque l'instance est bien structurée.

Temps de calcul. Les temps de calcul sont très faibles pour les petites grilles. Jusqu'à 6×6 , le temps moyen reste inférieur à 0.1 seconde. Pour 7×7 et 8×8 , le temps moyen reste autour de 0.25 seconde.

À partir de 9×9 , le temps augmente. Le temps moyen est d'environ 2.8 secondes en 9×9 , 4.9 secondes en 10×10 , puis 10.4 secondes sur les instances 11×11 testées. Pour 12×12 , le temps moyen des instances résolues dépasse 14 secondes.

Jusqu'à quelle taille la résolution est-elle possible ? Sur le dataset généré, CPLEX résout toutes les instances jusqu'à 8×8 . Il reste encore efficace sur les tailles 9×9 et 10×10 , mais les temps deviennent plus variables. Pour 12×12 , la résolution devient plus difficile.

Cependant, le test sur une vraie grille 20×20 montre que la taille seule n'est pas le seul facteur important. Une grande grille peut rester facile si les indices donnés guident suffisamment le solveur dans la reconstruction des régions.

Heuristique. L'heuristique obtient de bons résultats sur les petites instances. Elle résout toutes les instances en 3×3 , puis 39 instances sur 50 en 5×5 et 40 instances sur 50 en 6×6 .

Ensuite, ses performances diminuent : elle résout 12 instances sur 50 en 7×7 , 4 instances sur 50 en 8×8 , 5 instances sur 50 en 9×9 , et seulement 2 instances sur 50 en 10×10 . Sur la grille 20×20 , elle n'a pas réussi à trouver une solution.

Cela montre que l'heuristique est utile pour certaines petites grilles, mais qu'elle reste moins fiable que CPLEX.

Gap après 10 minutes. Cette question ne s'applique pas ici. Notre fonction objectif est constante.

4.5 Observations

- CPLEX trouve toutes les solutions pour les tailles de 3×3 à 8×8 .
- À partir de 9×9 , certaines instances deviennent beaucoup plus longues pour CPLEX.
- Pour 12×12 , plusieurs instances atteignent la limite de temps.
- Le test sur une vraie grille 20×20 montre que CPLEX peut aussi résoudre de grandes grilles bien structurées.
- L’heuristique est très rapide quand elle trouve une solution, mais elle échoue souvent sur les grandes tailles.
- Les instances non résolues ne sont pas forcément infaisables, car elles sont générées à partir de solutions connues.
- La difficulté vient surtout de la taille de la grille et de la structure des régions à reconstruire à partir de peu d’indices.
- Le modèle PLNE avec callback reste la méthode la plus fiable dans nos tests.

5 Conclusion

Filling se modélise naturellement comme un problème de programmation linéaire en nombres entiers. Les variables binaires permettent d’associer une valeur à chaque case, tandis que les chiffres déjà présents dans la grille sont fixés par des contraintes simples.

La règle difficile du jeu est la règle de connexité des blocs. Pour la gérer, nous utilisons un callback CPLEX. À chaque solution entière trouvée, le callback vérifie les composantes connexes de chaque valeur. Si une composante n’a pas la bonne taille, une contrainte paresseuse est ajoutée pour éliminer cette solution.

Les résultats montrent que CPLEX est la méthode la plus fiable dans nos tests. Il résout toutes les instances générées jusqu’à 8×8 , reste efficace sur plusieurs instances plus grandes, et parvient aussi à résoudre une vraie grille 20×20 .

Cependant, les temps de résolution augmentent lorsque la taille de la grille augmente. À partir de 9×9 , certaines instances demandent plus de temps, et en 12×12 , plusieurs instances atteignent la limite de temps. Les instances non résolues ne sont pas forcément infaisables, car elles sont générées à partir de solutions connues.

L’heuristique avec backtracking et vérifications locales donne de bons résultats sur certaines petites grilles, mais elle devient moins fiable lorsque la taille augmente. Elle reste donc utile comme comparaison, mais ne remplace pas la méthode exacte avec CPLEX.

Ainsi, la difficulté du jeu ne dépend pas seulement de la taille de la grille. Elle dépend aussi de la structure de l’instance, du nombre d’indices donnés et de la façon dont les régions doivent être reconstruites.

References

- [1] Simon Tatham's Portable Puzzle Collection. *Puzzles*. Disponible en ligne : <https://www.chiark.greenend.org.uk/~sgtatham/puzzles/>
- [2] Nikoli. *Fillomino*. Disponible en ligne : <https://www.nikoli.co.jp/en/puzzles/fillomino/>